# Environnement et Tooling

---

## Objectifs

1. Initialiser un projet moderne avec les **Go Modules** (`go.mod`).
2. Comprendre et corriger les erreurs strictes du compilateur Go et de son analyseur lexical (Lexer).
3. Utiliser la chaîne d'outils intégrée (`go run`, `go build`, `go fmt`, `go vet`).
4. Produire des exécutables binaires autonomes pour différentes architectures matérielles et systèmes d'exploitation (*Cross-compilation*).

---

## Exercice 1 : Initialisation et sonde système (`sysprobe`)

Vous devez créer un utilitaire en ligne de commande nommé `sysprobe` chargé d'afficher les caractéristiques de l'environnement d'exécution et du runtime Go lors du démarrage d'un conteneur ou d'un serveur.

### Travail demandé

1. Créez un répertoire dédié `tp-j1s1` et initialisez-y un module Go nommé `formation.go/sysprobe`.
2. Créez le fichier point d'entrée de votre exécutable.
3. À l'aide de la bibliothèque standard (explorez les packages `fmt`, `os` et `runtime`), écrivez un programme qui affiche dans la sortie standard :
    - La version exacte du compilateur/runtime Go qui a compilé le binaire.
    - Le système d'exploitation cible (`GOOS`) et l'architecture processeur (`GOARCH`).
    - Le nombre de cœurs logiques (CPU) détectés par le runtime sur la machine hôte.
    - Le répertoire de travail courant (*Working Directory*) depuis lequel le programme est lancé.
    - Le PID (*Process ID*) du processus en cours d'exécution.
4. Testez votre programme à la volée sans produire de fichier binaire persistant sur le disque, puis compilez-le proprement sous le nom `sysprobe` (ou `sysprobe.exe` sous Windows) et exécutez-le.

### Exemple de sortie attendue

```
None
=== SONDE SYSTÈME GO ===
Version Go       : go1.22.x
OS / Arch        : linux / amd64
CPUs disponibles : 8
Dossier courant  : /home/dev/tp-j1s1
PID du processus : 48291
```

---

## Exercice 2 : Le compilateur impitoyable et le piège du Lexer

Un développeur habitué au C et au Java vient d'écrire son premier fichier en Go. Il a également volontairement mal indenté son code et laissé des éléments de débogage.

### Travail demandé

1. Remplacez temporairement le contenu de votre fichier principal par le code ci-dessous (en conservant sa mise en forme dégradée) :

```go
package main

import (
    "fmt"
    "os"
    "time"
)

func main()
{
    var codeSortie int = 0
    nomApplication := "MonServeur"
    version := 1

    fmt.Printf("Démarrage de %s (v%s)...\n", nomApplication, version)
}
```

2. Tentez de compiler ce programme avec `go build`.
3. **Analysez et corrigez une par une** toutes les erreurs qui empêchent la compilation ou qui posent un problème de qualité :
    - **Étape A (Analyse lexicale) :** Pourquoi l'accolade ouvrante `{` de `func main()` provoque-t-elle une erreur `syntax error: unexpected semicolon or`

`newline before {` alors qu'aucun point-virgule n'est visible dans le code ? Corrigez le problème.

- **Étape B (Tolérance zéro sur le code mort) :** Identifiez les éléments refusés par le compilateur et nettoyez le code pour que `go build` accepte de compiler sans erreur.
- **Étape C (Analyse statique avec `go vet`) :** Une fois que le programme compile, il subsiste un bug subtil dans l'appel à `fmt.Printf` (qui provoquerait un affichage erroné à l'exécution). Exécutez la commande native d'analyse statique de Go pour faire ressortir l'anomalie, puis corrigez le verbe de formatage.
- **Étape D (Standardisation du style) :** Modifiez l'indentation du fichier de manière anarchique, puis utilisez la commande native de Go permettant de reformater automatiquement le fichier sur place selon la norme officielle.

---

## Exercice 3 : Cross-Compilation et inspection des binaires

Votre outil `sysprobe` de l'exercice 1 doit maintenant être déployé sur trois cibles matérielles différentes en production :

- Un serveur Cloud Linux en architecture `x86_64` (`amd64`).
- Une passerelle IoT industrielle sous Linux équipée d'un processeur ARM 64 bits (`arm64`, type Raspberry Pi 4/5 ou AWS Graviton).
- Un poste d'administration sous Windows 64 bits (`amd64`).

### Travail demandé

1. Remettez en place le code fonctionnel de votre sonde `sysprobe` (Exercice 1).
2. Sans installer de chaîne de compilation externe (*toolchain*) ni utiliser Docker, générez dans un sous-dossier `bin/` les trois exécutables suivants à partir de votre poste :
   - `bin/sysprobe-linux-amd64`
   - `bin/sysprobe-linux-arm64`
   - `bin/sysprobe-windows-amd64.exe`
3. **Analyse de la taille des binaires :**
   - Observez la taille (en Mo) des exécutables générés (`ls -lh bin/` sous Linux/macOS ou `dir bin\` sous Windows). Pourquoi un programme de 15 lignes pèse-t-il près de 2 Mo en Go contre quelques kilo-octets en C ?
   - Cherchez dans l'aide du compilateur (`go help build` et la documentation des flags de l'éditeur de liens `-ldflags`) comment supprimer la table des symboles et les informations de débogage DWARF lors de la compilation (`-s -w`).

- Recompilez `bin/sysprobe-linux-amd64-stripped` avec ces options d'optimisation et comparez le gain de taille obtenu.